ADMINISTRACIÓN DE SISTEMAS GESTORES DE BASES DE DATOS · 2º ASIR

Guardar objetos de Python en una base de datos relacional

Práctica guiada paso a paso: el desfase objeto-relacional y los ORM vistos desde el SGBD

Dato	Descripción
Duración estimada	2 sesiones de 55 minutos
Entorno	Máquina virtual con Ubuntu Server 24.04 y MariaDB 10.11
Herramientas	Python 3, SQLAlchemy 2 (ORM) y PyMySQL (conector)
Entrega	Este documento con las 24 capturas pegadas, exportado a PDF

1. Objetivos

- Entender qué es el desfase objeto-relacional y cómo lo resuelve un ORM.
- Guardar y recuperar objetos de Python en MariaDB sin escribir SQL en el programa.
- Comprobar desde el SGBD qué tablas y qué sentencias SQL genera el ORM.
- Aplicar el principio de mínimo privilegio al usuario de una aplicación.

2. Conceptos previos

El desfase objeto-relacional

Los programas modernos trabajan con **objetos**: un alumno es un objeto con atributos (nombre, apellidos, curso). Las bases de datos relacionales, en cambio, guardan **filas en tablas**. Son dos formas distintas de organizar la información, y pasar de una a otra a mano obliga a escribir mucho SQL repetitivo. A esa diferencia se le llama **desfase objeto-relacional**.

Qué es un ORM

Un **ORM** (Object-Relational Mapping, mapeo objeto-relacional) es una librería que hace esa traducción automáticamente: el programador trabaja con objetos y el ORM genera las sentencias SQL. En esta práctica se usa **SQLAlchemy**, el ORM más utilizado en Python.

En Python (objetos)	En la base de datos (relacional)
Clase	Tabla
Atributo	Columna
Objeto	Fila
Identificador del objeto	Clave primaria

Por qué interesa al administrador del SGBD

Aunque el ORM lo use el programador, quien administra el servidor debe saber qué SQL llega a la base de datos, qué tablas se crean y con qué permisos se conecta la aplicación. Por eso en esta práctica se prepara el servidor, se observa lo que hace el ORM y se controlan los permisos.

3. Antes de empezar

Necesitas una máquina virtual (por ejemplo en VirtualBox) con estas características:

Elemento	Valor recomendado
Sistema operativo	Ubuntu Server 24.04 LTS recién instalado
Memoria RAM	2 GB
Procesadores	2
Disco	20 GB
Red	Adaptador en NAT, con acceso a Internet
Usuario	Un usuario con permisos de sudo (en esta guía, alumno)

Convenciones de esta guía:

- Las órdenes que empiezan en el terminal de Linux se escriben tras el indicador `alumno@asir-server:~$`.
- Las sentencias SQL se escriben dentro de la consola de MariaDB, cuyo indicador es `MariaDB [...]>`. Todas terminan en punto y coma.
- Después de cada orden se pulsa `Intro`.

4. Desarrollo paso a paso

Realiza los pasos en orden. Cada vez que aparezca un recuadro de captura, haz una captura de pantalla de tu terminal en ese momento y pégala en el recuadro.

Paso 1. Comprobar la máquina virtual

1.1 Arranca la máquina virtual con Ubuntu Server 24.04 e inicia sesión con tu usuario (en esta guía se llama `alumno` y el equipo se llama `asir-server`).

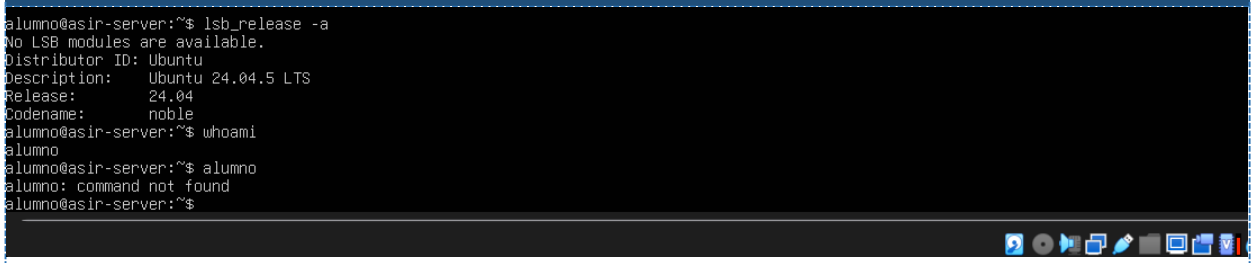
1.2 Comprueba la versión del sistema operativo con la orden:

```
lsb_release -a
```

1.3 Comprueba con qué usuario has iniciado sesión:

```
whoami
```

Captura 1. Versión del sistema y usuario con el que se trabaja.



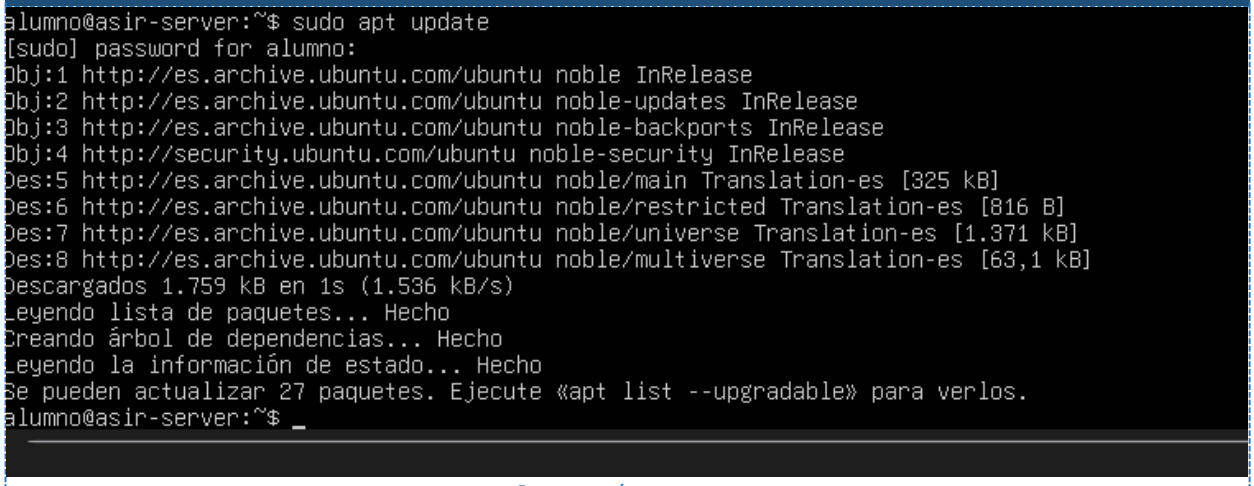
Pega aquí tu captura

Paso 2. Instalar MariaDB y las herramientas de Python

2.1 Actualiza la lista de paquetes disponibles. Te pedirá la contraseña de tu usuario; al escribirla no aparece nada en pantalla, es normal.

```
sudo apt update
```

Captura 2. Actualización de la lista de paquetes.

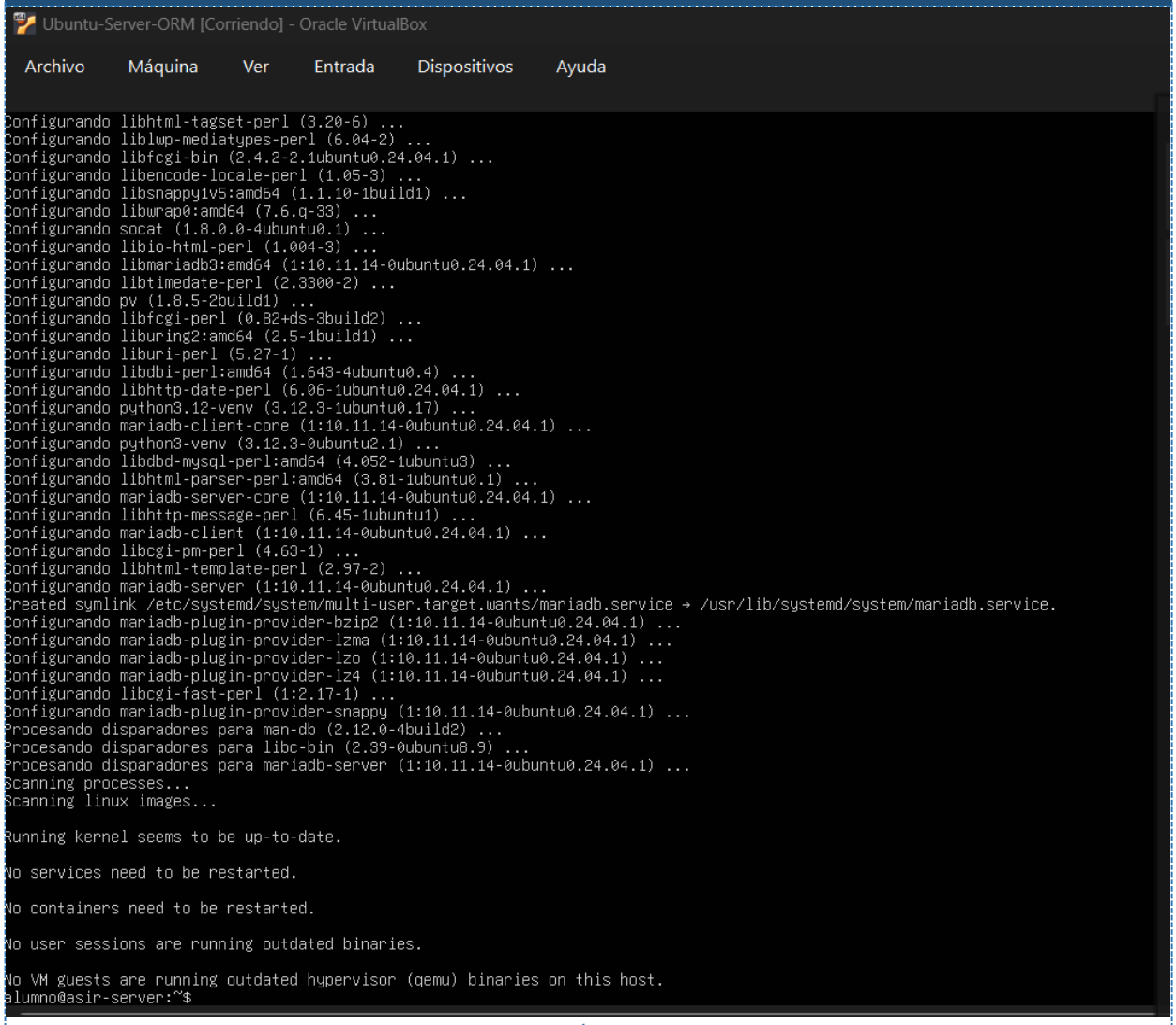


Pega aquí tu captura

2.2 Instala el servidor MariaDB y el módulo para crear entornos virtuales de Python. La opción `-y` responde que sí automáticamente a la confirmación.

```
sudo apt install -y mariadb-server python3-venv
```

Captura 3. Instalación de MariaDB y de python3-venv.

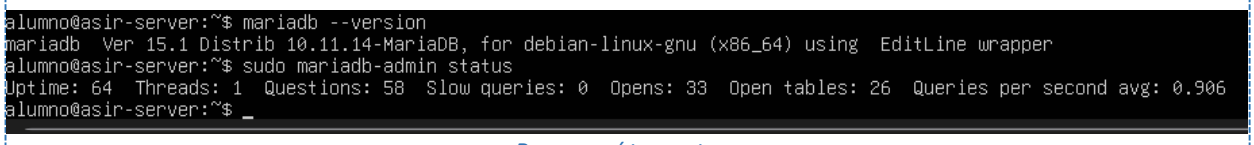


Pega aquí tu captura

2.3 Comprueba la versión del cliente de MariaDB y que el servidor está funcionando:

```
mariadb --version
sudo mariadb-admin status
```

Captura 4. Versión del cliente y estado del servidor MariaDB.



Pega aquí tu captura

Paso 3. Crear la base de datos

3.1 Entra en la consola de MariaDB como administrador. En Ubuntu el usuario root de MariaDB se autentica con el usuario root del sistema, por eso se usa `sudo` y no se pide contraseña de MariaDB:

```
sudo mariadb
```

3.2 El indicador cambia a `MariaDB [(none)]>`. Crea la base de datos `instituto` con el juego de caracteres `utf8mb4`, que admite tildes, eñes y cualquier carácter:

```
CREATE DATABASE instituto CHARACTER SET utf8mb4 COLLATE utf8mb4_general_ci;
```

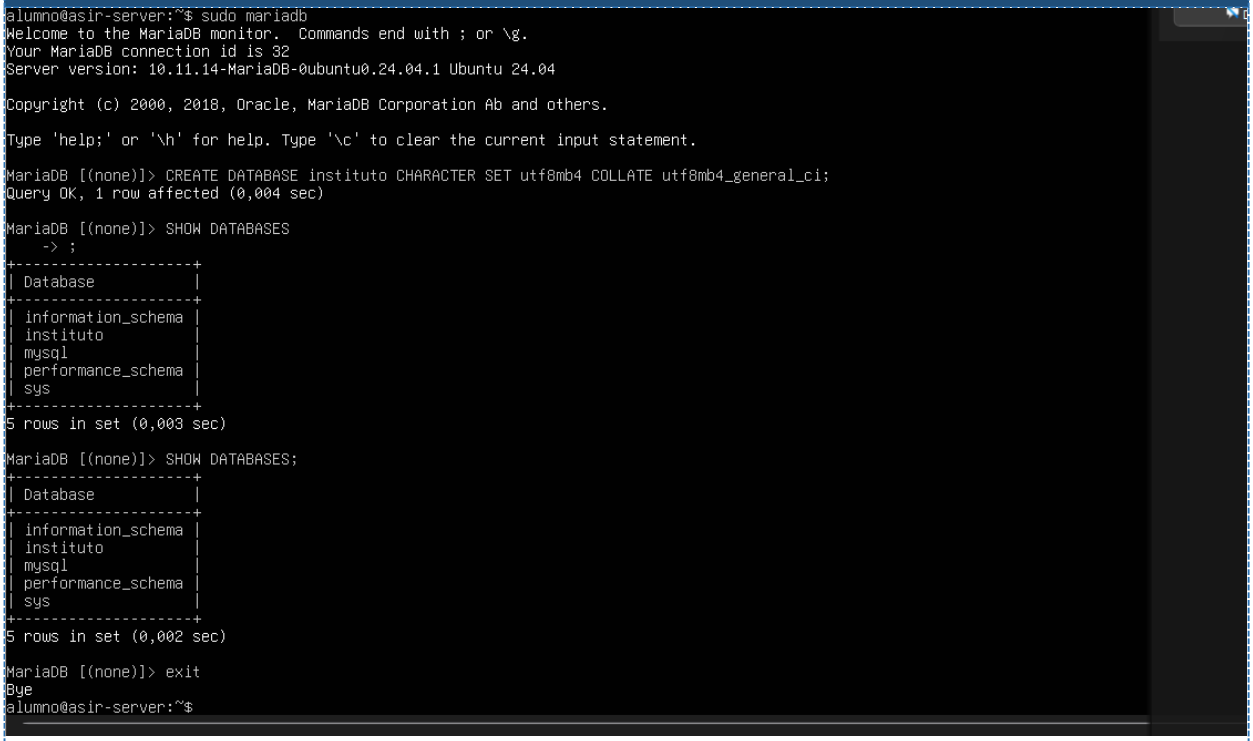
3.3 Comprueba que aparece en la lista de bases de datos:

```
SHOW DATABASES;
```

3.4 Sal de la consola de MariaDB:

```
exit
```

Captura 5. Creación de la base de datos instituto.



Pega aquí tu captura

Paso 4. Crear el usuario de la aplicación

Una aplicación nunca debe conectarse a la base de datos como `root`. Se crea un usuario propio que solo tiene los privilegios que necesita.

4.1 Vuelve a entrar como administrador:

```
sudo mariadb
```

4.2 Crea el usuario `app_instituto`, que solo podrá conectarse desde el propio servidor (`localhost`), con la contraseña `Asir2026*`:

```
CREATE USER 'app_instituto'@'localhost' IDENTIFIED BY 'Asir2026*';
```

4.3 Concédele solo los privilegios de consultar, insertar, modificar, borrar filas y crear tablas, y únicamente sobre la base de datos `instituto`:

```
GRANT SELECT, INSERT, UPDATE, DELETE, CREATE ON instituto.* TO
'app_instituto'@'localhost';
```

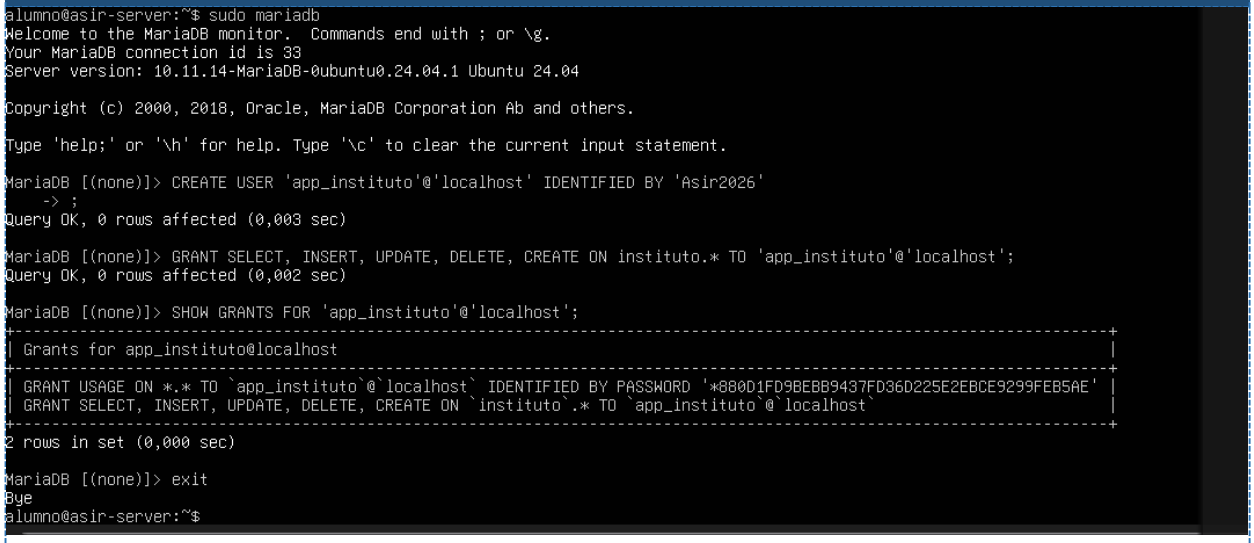
4.4 Muestra los privilegios del usuario para comprobarlos:

```
SHOW GRANTS FOR 'app_instituto'@'localhost';
```

4.5 Sal de la consola:

```
exit
```

Captura 6. Creación del usuario app_instituto y sus privilegios.



Pega aquí tu captura

Paso 5. Activar el registro general de consultas

El registro general (general log) guarda todas las sentencias que recibe el servidor. Servirá para ver exactamente qué SQL escribe el ORM a partir de los objetos de Python.

5.1 Entra como administrador:

```
sudo mariadb
```

5.2 Indica en qué fichero se guardará el registro y actívalo:

```
SET GLOBAL general_log_file = '/var/lib/mysql/consultas.log';
SET GLOBAL general_log = ON;
```

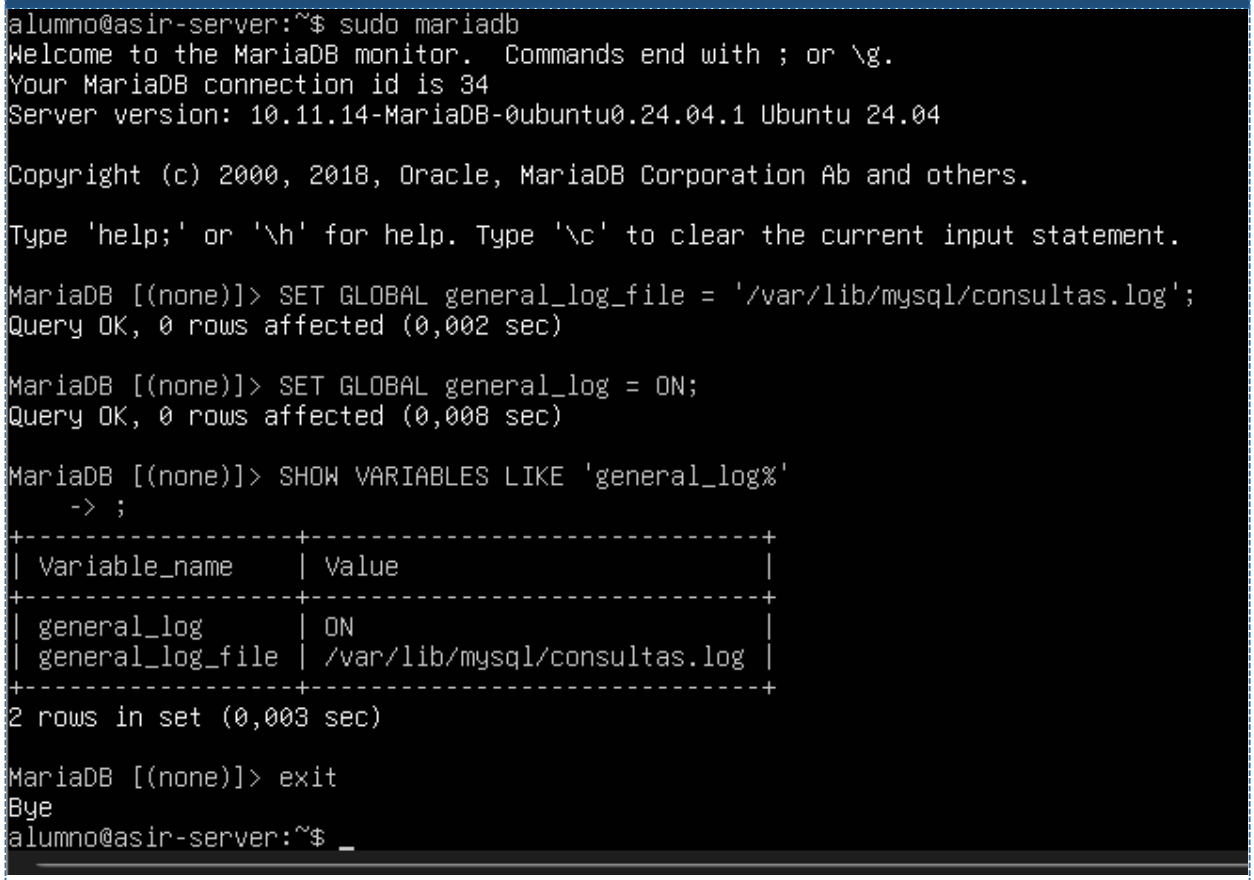
5.3 Comprueba el valor de las dos variables:

```
SHOW VARIABLES LIKE 'general_log%';
```

5.4 Sal de la consola:

```
exit
```

Captura 7. Activación del registro general de consultas.



Pega aquí tu captura

Nota. este registro solo se activa durante la práctica. En un servidor en producción consume disco y rendimiento, por eso se desactiva en el último paso.

Paso 6. Preparar el entorno de Python

Ubuntu 24.04 no permite instalar librerías con pip directamente en el sistema, así que se trabaja dentro de un entorno virtual: una carpeta con su propio Python y sus propias librerías.

6.1 Crea la carpeta de la práctica y entra en ella:

```
mkdir ~/practica_orm
cd ~/practica_orm
```

6.2 Crea el entorno virtual, que se llamará `venv`, y comprueba que aparece su carpeta:

```
python3 -m venv venv
ls
```

6.3 Activa el entorno virtual. El indicador del terminal empezará por `(venv)`:

```
source venv/bin/activate
```

Captura 8. Creación y activación del entorno virtual.

```
alumno@asir-server:~$ cd ~/practica_orm+
-bash: cd: /home/alumno/practica_orm+: No such file or directory
alumno@asir-server:~$ cd ~/practica_orm
alumno@asir-server:~/practica_orm$ python3 -m venv venv
alumno@asir-server:~/practica_orm$ ls
venv
alumno@asir-server:~/practica_orm$ source venv/bin/activate
(venv) alumno@asir-server:~/practica_orm$
```

Pega aquí tu captura

6.4 Instala el ORM (SQLAlchemy) y el conector que habla con MariaDB (PyMySQL):

```
pip install sqlalchemy pymysql
```

Captura 9. Instalación de SQLAlchemy y PyMySQL.

```
(venv) alumno@asir-server:~/practica_orm$ pip install sqlalchemy pymysql
Collecting sqlalchemy
  Downloading sqlalchemy-2.1.6-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata
 (9.7 kB)
Collecting pymysql
  Downloading pymysql-1.2.3-py3-none-any.whl.metadata (4.2 kB)
Collecting typing-extensions>=4.6.0 (from sqlalchemy)
  Downloading typing_extensions-4.16.0-py3-none-any.whl.metadata (3.3 kB)
Downloading sqlalchemy-2.1.6-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (4.7 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 4.7/4.7 MB 2.9 MB/s eta 0:00:00
Downloading pymysql-1.2.3-py3-none-any.whl (46 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 46.7/46.7 kB 2.7 MB/s eta 0:00:00
Downloading typing_extensions-4.16.0-py3-none-any.whl (45 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 45.6/45.6 kB 703.0 kB/s eta 0:00:00
Installing collected packages: typing-extensions, pymysql, sqlalchemy
Successfully installed pymysql-1.2.3 sqlalchemy-2.1.6 typing-extensions-4.16.0
(venv) alumno@asir-server:~/practica_orm$
```

Pega aquí tu captura

6.5 Comprueba la versión de SQLAlchemy y los paquetes instalados en el entorno:

```
python -c "import sqlalchemy; print(sqlalchemy.__version__)"
pip list
```

Captura 10. Versión de SQLAlchemy y paquetes del entorno virtual.

```
(venv) alumno@asir-server:~/practica_orm$ python -c "import sqlalchemy; print(sqlalchemy.__version__)"
2.1.6
(venv) alumno@asir-server:~/practica_orm$ pip list
Package           Version
----------------- -------
pip               24.0
PyMySQL           1.2.3
SQLAlchemy        2.1.6
typing_extensions 4.16.0
(venv) alumno@asir-server:~/practica_orm$
```

Pega aquí tu captura

Nota. todos los pasos siguientes se hacen dentro de la carpeta `~/practica_orm` y con el entorno virtual activo. Si cierras el terminal, vuelve a la carpeta y ejecuta otra vez `source venv/bin/activate`.

Paso 7. Crear el modelo: la clase Alumno

Este es el núcleo del ORM: una clase de Python que se corresponde con una tabla de la base de datos.

7.1 Abre el editor nano con un fichero nuevo:

```
nano modelo.py
```

7.2 Escribe exactamente este código. Respeta los espacios del principio de las líneas: en Python indican qué líneas pertenecen a cada clase.

```
from sqlalchemy import create_engine, Integer, String
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column

# Cadena de conexión: dialecto+driver://usuario:contraseña@servidor/base_de_datos
URL = "mysql+pymysql://app_instituto:Asir2026*@localhost/instituto?charset=utf8mb4"
engine = create_engine(URL)

class Base(DeclarativeBase):
    pass

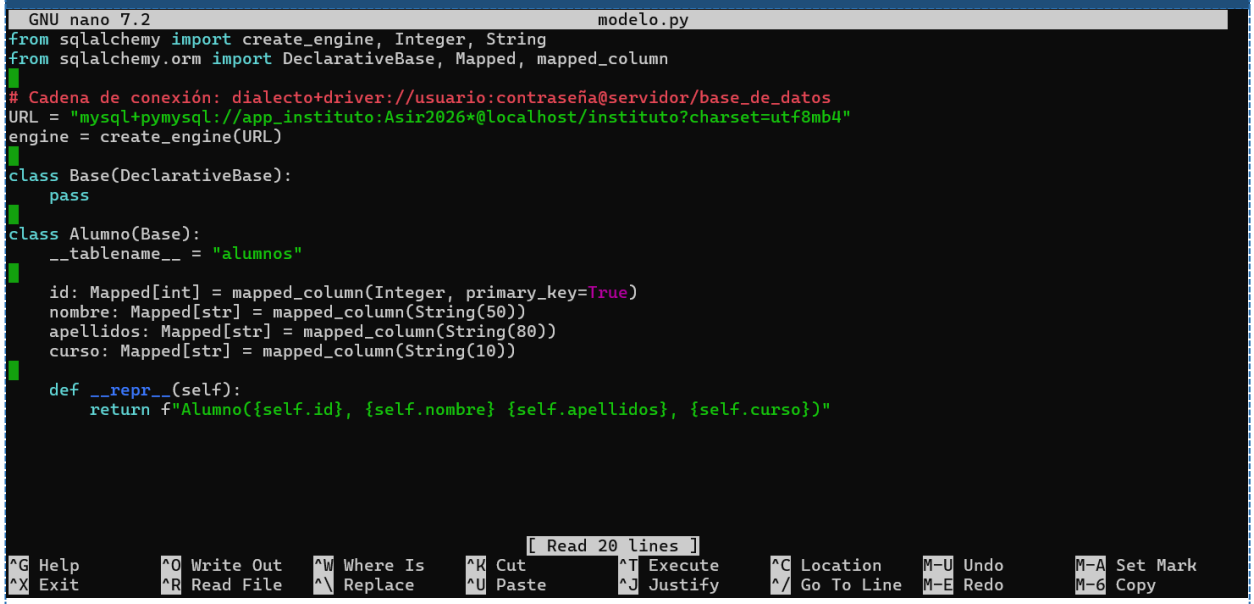
class Alumno(Base):
    __tablename__ = "alumnos"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    nombre: Mapped[str] = mapped_column(String(50))
    apellidos: Mapped[str] = mapped_column(String(80))
    curso: Mapped[str] = mapped_column(String(10))

    def __repr__(self):
        return f"Alumno({self.id}, {self.nombre} {self.apellidos}, {self.curso})"
```

7.3 Guarda con `Ctrl+O` y pulsa `Intro` para confirmar el nombre. Abajo aparecerá el mensaje `[ Wrote 20 lines ]`.

Captura 11. Fichero modelo.py guardado en nano.



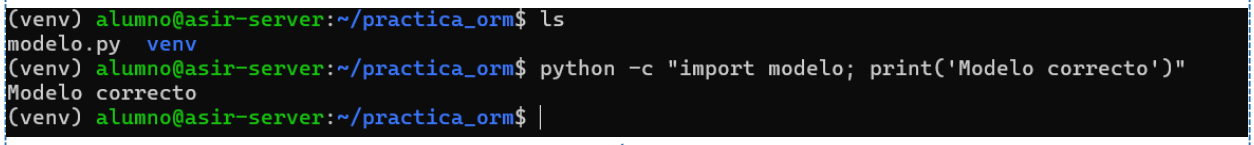
Pega aquí tu captura

7.4 Sal de nano con `Ctrl+X`.

7.5 Comprueba que el fichero existe y que Python lo importa sin errores:

```
ls
python -c "import modelo; print('Modelo correcto')"
```

Captura 12. Comprobación de que el modelo se importa sin errores.



Pega aquí tu captura

Qué hace cada parte del código:

- `URL` contiene los datos de conexión: el usuario, la contraseña y la base de datos creados en los pasos 3 y 4.
- `engine` es el objeto que gestiona la conexión con MariaDB.
- `Base` es la clase de la que heredan todas las clases que se guardarán en la base de datos.
- `__tablename__` es el nombre de la tabla. Cada atributo con `mapped_column` será una columna; `id` será la clave primaria.
- `__repr__` solo sirve para que el objeto se muestre de forma legible al imprimirlo.

Paso 8. Crear la tabla a partir de la clase

No vas a escribir ningún CREATE TABLE: el ORM lo genera a partir de la clase.

8.1 Crea el fichero con nano:

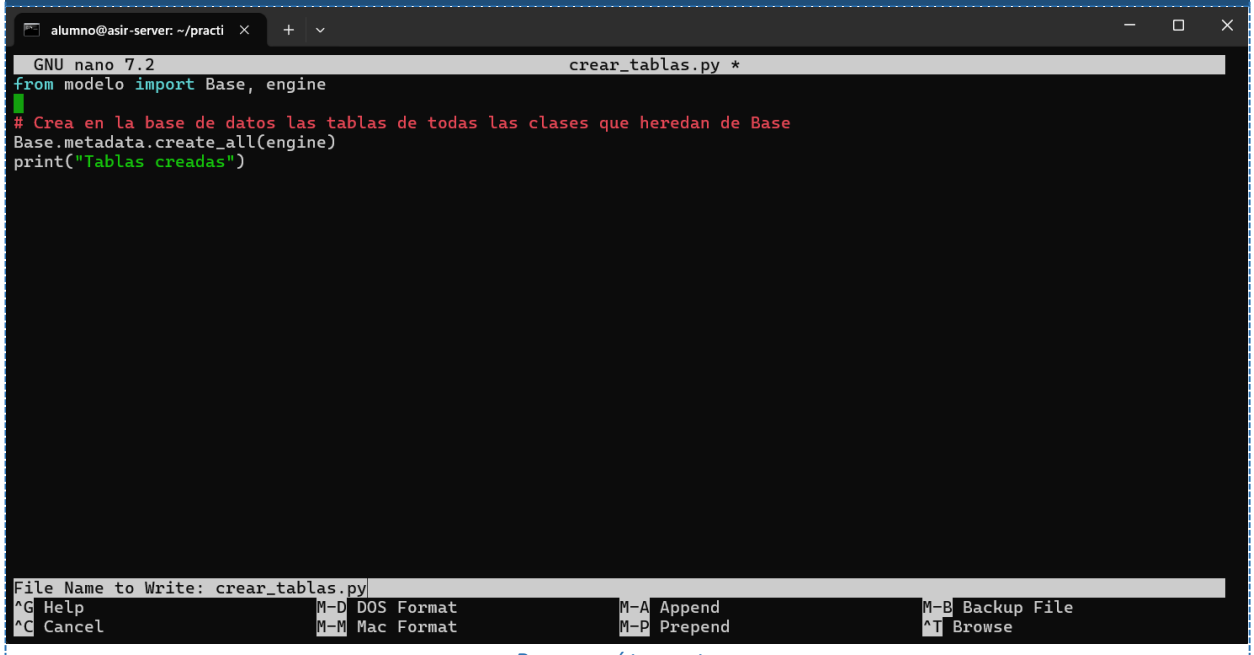
```
nano crear_tablas.py
```

8.2 Escribe este código, guarda con `Ctrl+O` e `Intro`, y sal con `Ctrl+X`:

```
from modelo import Base, engine

# Crea en la base de datos las tablas de todas las clases que heredan de Base
Base.metadata.create_all(engine)
print("Tablas creadas")
```

Captura 13. Fichero crear_tablas.py guardado en nano.

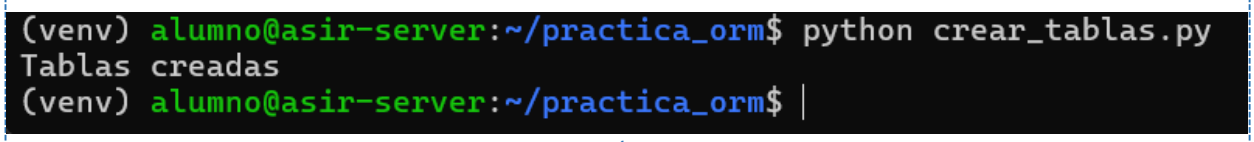


Pega aquí tu captura

8.3 Ejecuta el script:

```
python crear_tablas.py
```

Captura 14. Ejecución de crear_tablas.py.



Pega aquí tu captura

Paso 9. Comprobar la tabla desde MariaDB

Ahora miras el resultado como administrador del SGBD, sin Python.

9.1 Conéctate con el usuario de la aplicación directamente a la base de datos `instituto`. Cuando pida la contraseña escribe `Asir2026*` (no se verá al escribirla):

```
mariadb -u app_instituto -p instituto
```

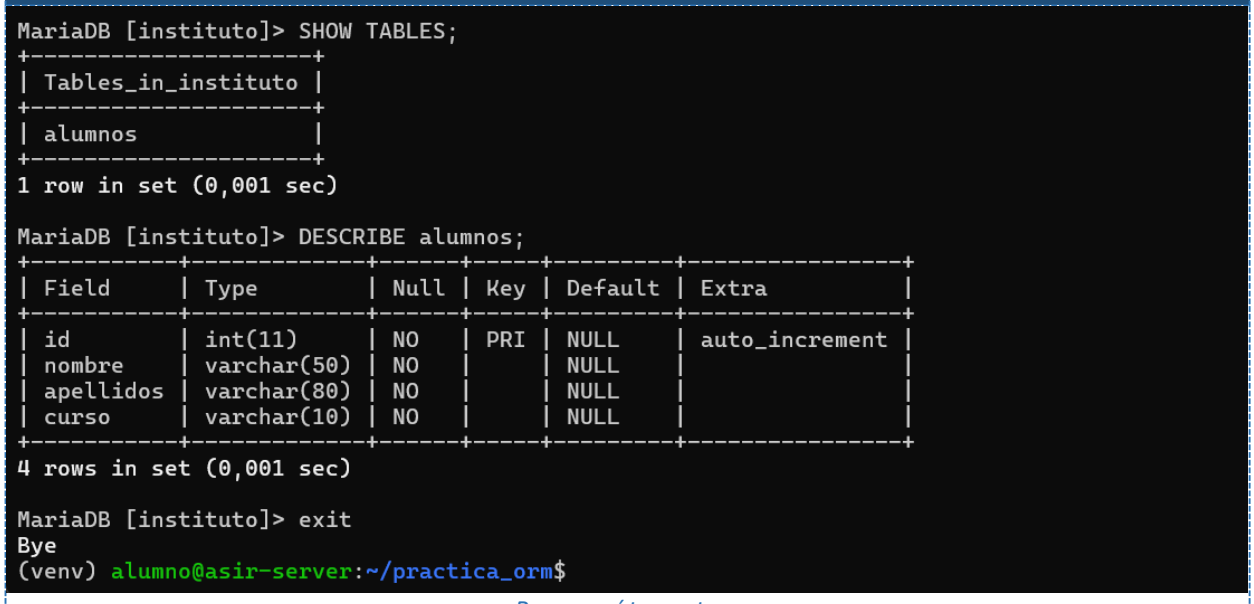
9.2 El indicador ahora es `MariaDB [instituto]>`. Lista las tablas y muestra la estructura de `alumnos`:

```
SHOW TABLES;
DESCRIBE alumnos;
```

9.3 Sal de la consola:

```
exit
```

Captura 15. Tabla alumnos creada por el ORM, vista desde MariaDB.



Pega aquí tu captura

9.4 Compara cada columna con los atributos de la clase Alumno del paso 7: nombre, tipo, si admite nulos y clave primaria.

Paso 10. Guardar objetos en la base de datos

Vas a crear objetos de Python y guardarlos sin escribir ningún INSERT.

10.1 Crea el fichero con nano:

```
nano insertar.py
```

10.2 Escribe este código, guarda con `Ctrl+O` e `Intro`, y sal con `Ctrl+X`:

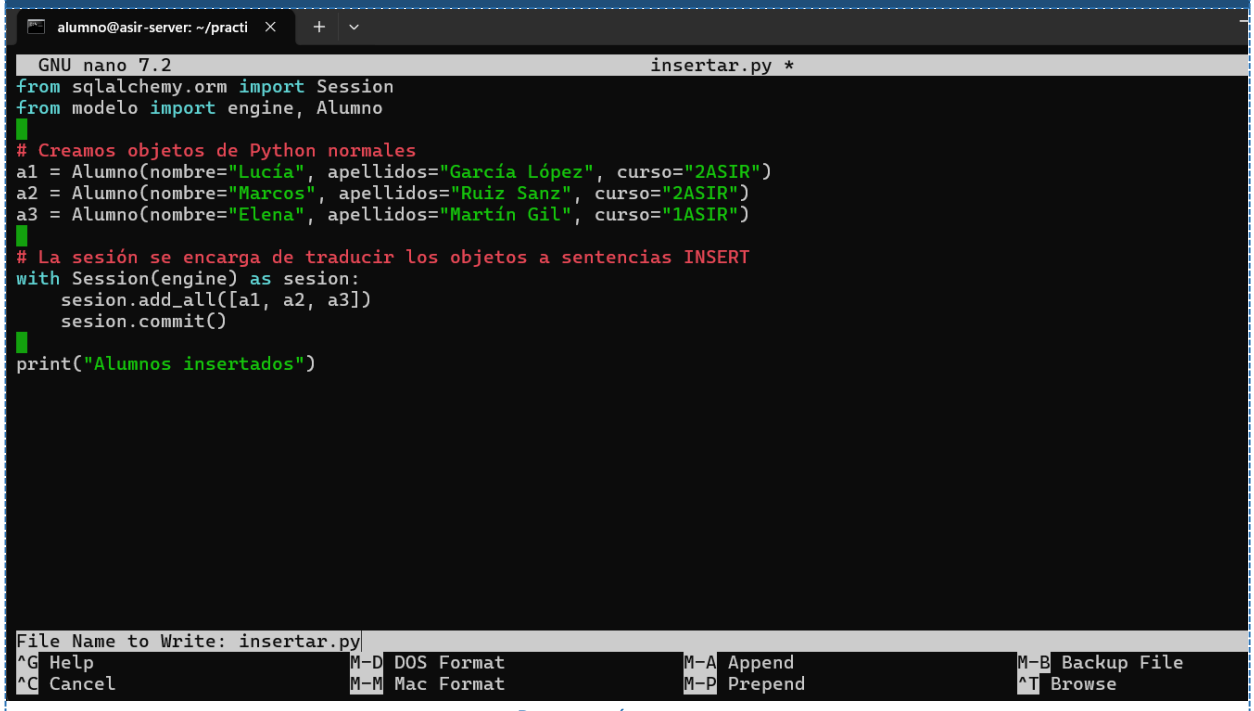
```
from sqlalchemy.orm import Session
from modelo import engine, Alumno

# Creamos objetos de Python normales
a1 = Alumno(nombre="Lucía", apellidos="García López", curso="2ASIR")
a2 = Alumno(nombre="Marcos", apellidos="Ruiz Sanz", curso="2ASIR")
a3 = Alumno(nombre="Elena", apellidos="Martín Gil", curso="1ASIR")

# La sesión se encarga de traducir los objetos a sentencias INSERT
with Session(engine) as sesion:
    sesion.add_all([a1, a2, a3])
    sesion.commit()

print("Alumnos insertados")
```

Captura 16. Fichero insertar.py guardado en nano.

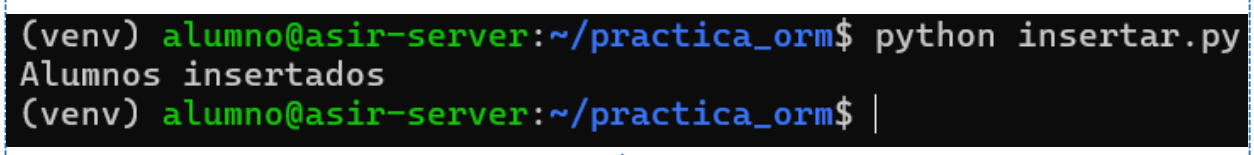


Pega aquí tu captura

10.3 Ejecuta el script una sola vez:

```
python insertar.py
```

Captura 17. Ejecución de insertar.py.



Pega aquí tu captura

Nota. si ejecutas este script más de una vez, se insertarán de nuevo los tres alumnos con identificadores nuevos.

Paso 11. Recuperar los objetos desde Python

Ahora el camino inverso: leer filas de la tabla y obtener objetos Alumno.

11.1 Crea el fichero con nano:

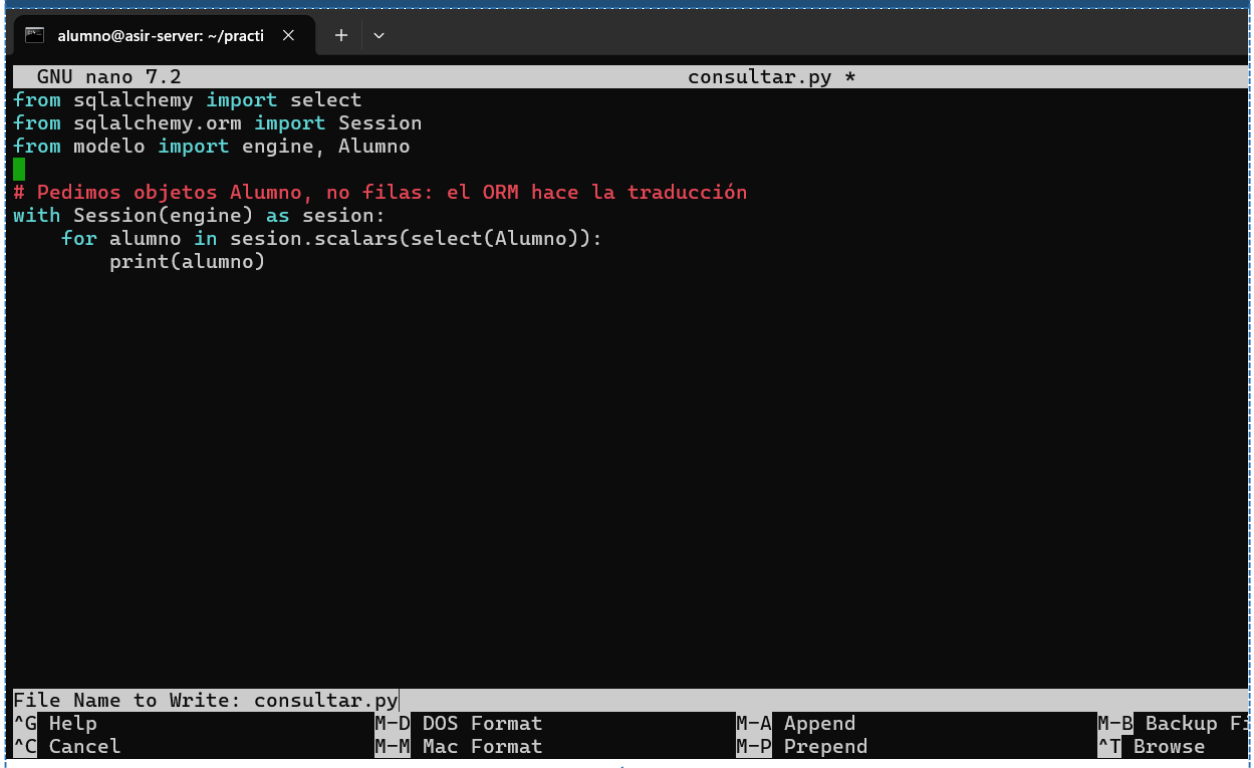
```
nano consultar.py
```

11.2 Escribe este código, guarda con `Ctrl+O` e `Intro`, y sal con `Ctrl+X`:

```
from sqlalchemy import select
from sqlalchemy.orm import Session
from modelo import engine, Alumno

# Pedimos objetos Alumno, no filas: el ORM hace la traducción
with Session(engine) as sesion:
    for alumno in sesion.scalars(select(Alumno)):
        print(alumno)
```

Captura 18. Fichero consultar.py guardado en nano.

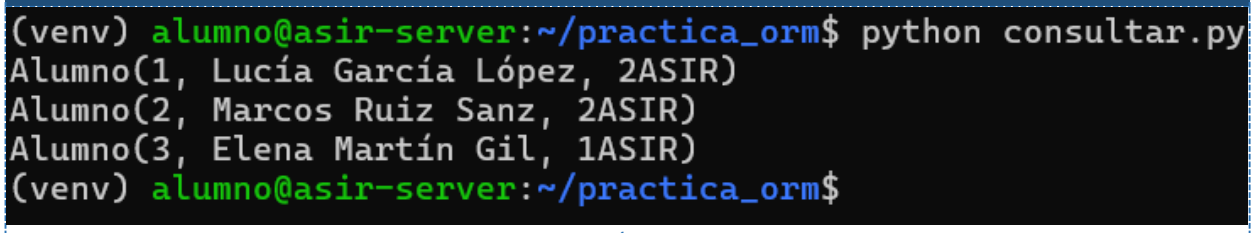


Pega aquí tu captura

11.3 Ejecuta el script:

```
python consultar.py
```

Captura 19. Objetos Alumno recuperados desde Python.



Pega aquí tu captura

Paso 12. Comprobar los datos desde MariaDB

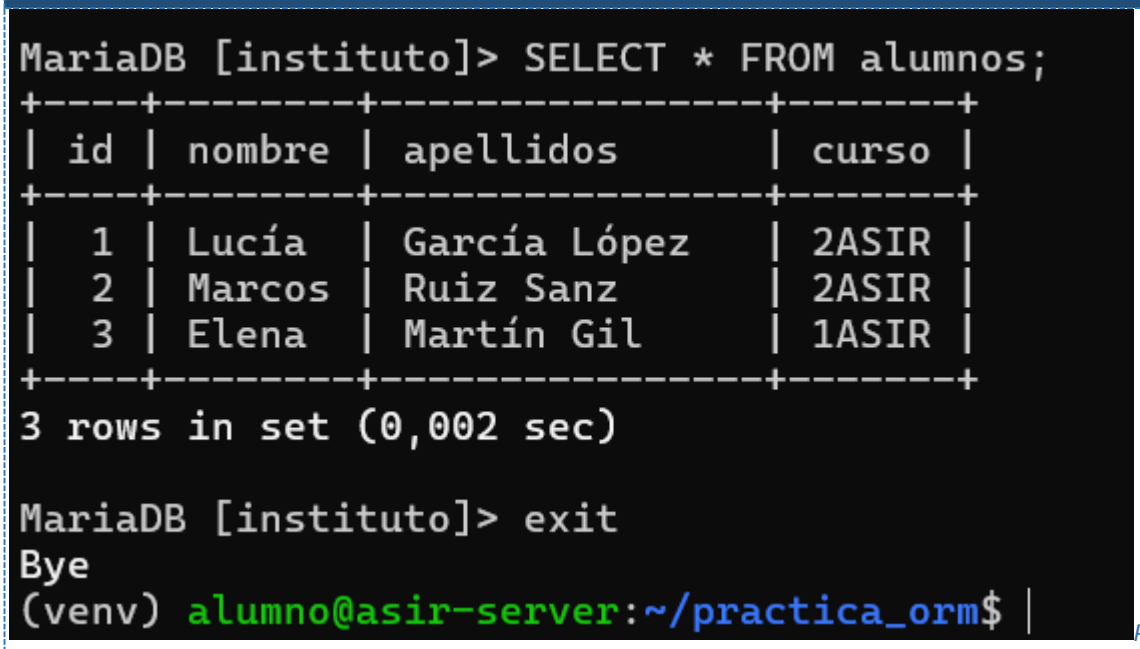
12.1 Conéctate de nuevo con el usuario de la aplicación (contraseña `Asir2026*`):

```
mariadb -u app_instituto -p instituto
```

12.2 Muestra todas las filas de la tabla y sal:

```
SELECT * FROM alumnos;
exit
```

Captura 20. Filas de la tabla alumnos vistas desde MariaDB.



Pega aquí tu captura

12.3 Comprueba que los datos coinciden con los objetos creados en el paso 10 y que las tildes se ven bien.

Paso 13. Ver el SQL que ha generado el ORM

El registro activado en el paso 5 contiene todas las sentencias que ha recibido el servidor, también las que escribió el ORM. El fichero pertenece al usuario del sistema `mysql`, por eso se lee con `sudo`.

13.1 Busca la sentencia que creó la tabla. La opción `-A 6` muestra también las 6 líneas siguientes, porque la sentencia ocupa varias líneas:

```
sudo grep -A 6 "CREATE TABLE" /var/lib/mysql/consultas.log
```

Captura 21. Sentencia CREATE TABLE generada por el ORM.

```
(venv) alumno@asir-server:~/practica_orm$ sudo grep -A 6 "CREATE TABLE" /var/lib/mysql/consultas.log
(venv) alumno@asir-server:~/practica_orm$ python -c "from modelo import Base, engine, Alumno; Alumno.__table__.to_metada
ta(__import__('sqlalchemy').MetaData(), name='alumnos_prueba').create(engine)"
(venv) alumno@asir-server:~/practica_orm$ sudo grep -A 6 "CREATE TABLE" /var/lib/mysql/consultas.log
                    45 Query    CREATE TABLE alumnos_prueba (
        id INTEGER NOT NULL AUTO_INCREMENT,
        nombre VARCHAR(50) NOT NULL,
        apellidos VARCHAR(80) NOT NULL,
        curso VARCHAR(10) NOT NULL,
        PRIMARY KEY (id)
)
(venv) alumno@asir-server:~/practica_orm$
```

Pega aquí tu captura

13.2 Busca la sentencia que insertó los alumnos y la consulta que los leyó:

```
sudo grep -A 1 "INSERT INTO" /var/lib/mysql/consultas.log
sudo grep -A 1 "SELECT alumnos" /var/lib/mysql/consultas.log
```

Captura 22. Sentencias INSERT y SELECT generadas por el ORM.

```
(venv) alumno@asir-server:~/practica_orm$ python -c "from sqlalchemy import insert; from modelo import engine, Alumno; t
=Alumno.__table__.to_metadata(__import__('sqlalchemy').MetaData(), name='alumnos_prueba'); c=engine.connect(); c.execute
(insert(t).values(nombre='Prueba', apellidos='ORM', curso='2ASIR')); c.commit(); c.close()"
(venv) alumno@asir-server:~/practica_orm$ sudo grep -A 1 "INSERT INTO" /var/lib/mysql/consultas.log
                    46 Query    INSERT INTO alumnos_prueba (nombre, apellidos, curso) VALUES ('Prueba', 'ORM', '2ASIR')
                    46 Query    COMMIT
(venv) alumno@asir-server:~/practica_orm$ sudo grep -A 1 "SELECT alumnos" /var/lib/mysql/consultas.log
                    43 Query    SELECT alumnos.id, alumnos.nombre, alumnos.apellidos, alumnos.curso
FROM alumnos
(venv) alumno@asir-server:~/practica_orm$
```

Pega aquí tu captura

Paso 14. Comprobar que el usuario tiene privilegios limitados

14.1 Conéctate con el usuario de la aplicación (contraseña `Asir2026*`):

```
mariadb -u app_instituto -p instituto
```

14.2 Intenta borrar la tabla y, a continuación, comprueba que sigue intacta:

```
DROP TABLE alumnos;
SELECT COUNT(*) FROM alumnos;
```

14.3 Sal de la consola:

```
exit
```

Captura 23. El usuario de la aplicación no puede borrar la tabla.

```
MariaDB [instituto]> DROP TABLE alumnos;
ERROR 1142 (42000): DROP command denied to user 'app_instituto'@'localhost' for table `instituto`.`alumnos`
MariaDB [instituto]> SELECT COUNT(*) FROM alumnos;
+----------+
| COUNT(*) |
+----------+
|        3 |
+----------+
1 row in set (0,001 sec)

MariaDB [instituto]> exit
Bye
(venv) alumno@asir-server:~/practica_orm$
```

Pega aquí tu captura

Paso 15. Desactivar el registro y cerrar

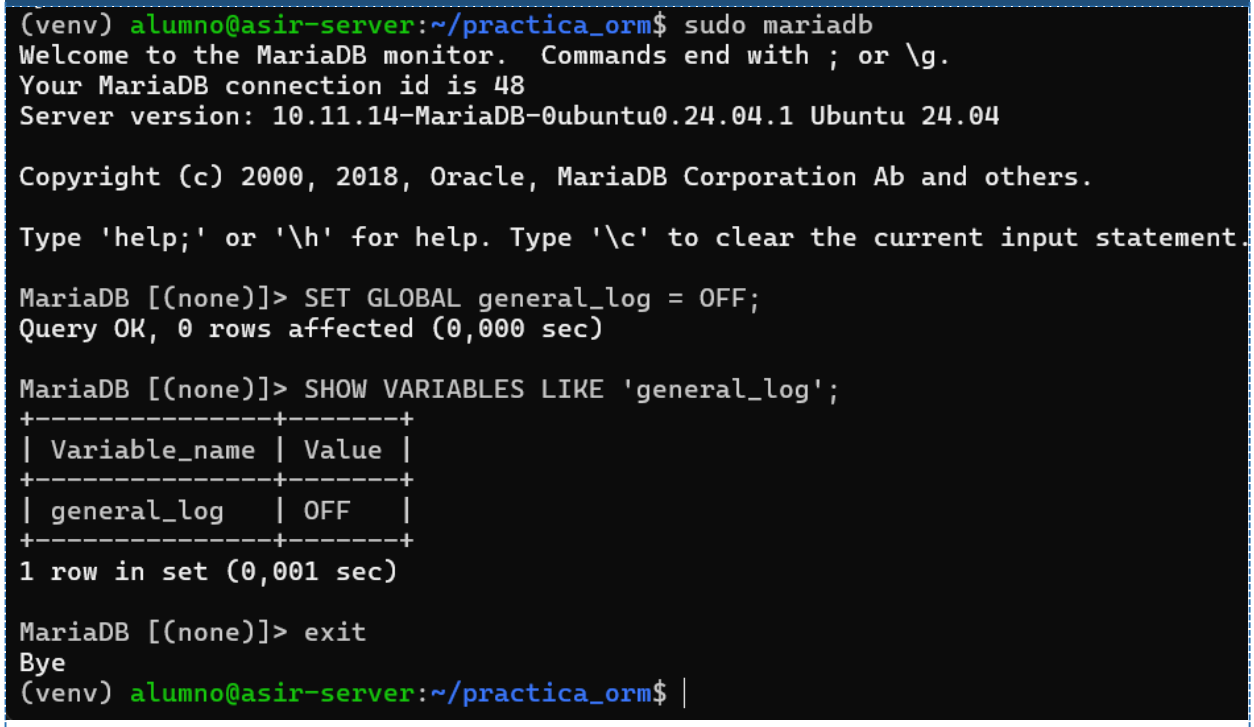
15.1 Entra como administrador, desactiva el registro general y comprueba su valor:

```
sudo mariadb
SET GLOBAL general_log = OFF;
SHOW VARIABLES LIKE 'general_log';
exit
```

15.2 Desactiva el entorno virtual de Python. El indicador dejará de mostrar `(venv)`:

```
deactivate
```

Captura 24. Desactivación del registro y salida del entorno virtual.



Pega aquí tu captura

5. Entrega

Entrega este mismo documento con las 24 capturas pegadas en sus recuadros, exportado a PDF con el nombre `Apellidos_Nombre_ORM.pdf`. En cada captura debe verse el indicador del terminal con tu usuario.